

Part II: The Async Order Processing System

Architecture Overview

Our system processes e-commerce orders using two patterns: synchronous (blocking) and asynchronous (event-driven). The architecture consists of:

- **Order Receiver** (ECS Fargate): Exposes `/orders/sync` and `/orders/async` endpoints behind an ALB
- **SNS Topic** (`CS6650L2-order-events`): Receives order events from the async endpoint
- **SQS Queue** (`CS6650L2-order-queue`): Buffers messages until workers are ready
- **Order Processor** (ECS Fargate): Polls SQS and processes orders concurrently

[Locust] → [ALB] → [ECS Receiver] → [SNS] → [SQS] → [ECS Processor]

↓
/orders/sync (blocks 3s)
/orders/async (returns 202 immediately)

Infrastructure: VPC (10.0.0.0/16), public subnets (10.0.1.0/24, 10.0.2.0/24) for ALB, private subnets (10.0.10.0/24, 10.0.11.0/24) for ECS, NAT Gateway for outbound AWS API access.

Phase 1 & 2: Synchronous Bottleneck

Normal Load Test (5 users, 30s)

```
(base) emilychen@emilys-Air-2 locust % TEST_MODE=sync locust -f locustfile.py \
--host=http://CS6650L2-alb-1613587836.us-west-2.elb.amazonaws.com \
--users=5 --spawn-rate=1 --run-time=30s --headless
Aggregated 7 0(0.00%) | 9292 3057 14740 9100 | 0.40 0.00

Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
POST POST /orders/sync 7 0(0.00%) | 9292 3057 14740 9100 | 0.40 0.00
Aggregated 7 0(0.00%) | 9292 3057 14740 9100 | 0.40 0.00

Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
POST POST /orders/sync 8 0(0.00%) | 9955 3057 14740 9100 | 0.40 0.00
Aggregated 8 0(0.00%) | 9955 3057 14740 9100 | 0.40 0.00

Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
POST POST /orders/sync 9 0(0.00%) | 10498 3057 14843 12000 | 0.40 0.00
Aggregated 9 0(0.00%) | 10498 3057 14843 12000 | 0.40 0.00

[2026-03-13 22:56:21,703] emilys-Air-2/INFO/locust.main: --run-time limit reached, shutting down
[2026-03-13 22:56:21,741] emilys-Air-2/INFO/locust.main: Shutting down (exit code 0)
Type Name # reqs # fails | Avg Min Max Med | req/s failures/s
POST POST /orders/sync 9 0(0.00%) | 10498 3057 14843 12000 | 0.33 0.00
Aggregated 9 0(0.00%) | 10498 3057 14843 12000 | 0.33 0.00

Response time percentiles (approximated)
Type Name 50% 66% 75% 80% 90% 95% 98% 99% 99.9% 99.99% 100% # reqs
POST POST /orders/sync 12000 14000 15000 15000 15000 15000 15000 15000 15000 15000 15000 9
Aggregated 12000 14000 15000 15000 15000 15000 15000 15000 15000 15000 15000 9

(base) emilychen@emilys-Air-2 locust %
```

Metric	Value
Requests	9
Failures	0%
Avg response	10,498ms
Min response	3,057ms
Max response	14,843ms

Even under normal load, response times ranged from 3s to 15s. The minimum of 3,057ms reflects the payment verification delay for the first user; subsequent users queued behind the single payment semaphore, waiting up to 15 seconds.

Flash Sale Load Test (20 users, 60s)

```
(base) emilychen@emilys-Air-2 locust % TEST_MODE=sync locust -f locustfile.py \
--host=http://CS6650L2-alb-1613587836.us-west-2.elb.amazonaws.com \
--users=20 --spawn-rate=10 --run-time=60s --headless

-----
| Aggregated | 17 | 0(0.00%) | 26676 | 3117 | 50133 | 27000 | 0.40 | 0.00 |
-----
Type  Name                                     # reqs  # fails  Avg    Min    Max    Med    req/s  failures/s
-----
POST  POST /orders/sync                          18      0(0.00%) 28144  3117  53110  27000  0.40   0.00
-----
| Aggregated | 18 | 0(0.00%) | 28144 | 3117 | 53110 | 27000 | 0.40 | 0.00 |
-----
Type  Name                                     # reqs  # fails  Avg    Min    Max    Med    req/s  failures/s
-----
POST  POST /orders/sync                          19      0(0.00%) 29614  3117  56073  30000  0.40   0.00
-----
| Aggregated | 19 | 0(0.00%) | 29614 | 3117 | 56073 | 30000 | 0.40 | 0.00 |
-----

[2026-03-13 23:00:31,190] emilys-Air-2/INFO/locust.main: --run-time limit reached, shutting down
[2026-03-13 23:00:31,244] emilys-Air-2/INFO/locust.main: Shutting down (exit code 0)
Type  Name                                     # reqs  # fails  Avg    Min    Max    Med    req/s  failures/s
-----
POST  POST /orders/sync                          19      0(0.00%) 29614  3117  56073  30000  0.33   0.00
-----
| Aggregated | 19 | 0(0.00%) | 29614 | 3117 | 56073 | 30000 | 0.33 | 0.00 |
-----

Response time percentiles (approximated)
Type  Name                                     % 100% # reqs  50%  66%  75%  80%  90%  95%  98%  99%  99.9%  99.99%
-----
POST  POST /orders/sync 0 56000 19 30000 38000 44000 47000 53000 56000 56000 56000 56000 56000
-----
| Aggregated | 19 | 30000 38000 44000 47000 53000 56000 56000 56000 56000 56000 |
-----
```

Metric	Value
Requests	19
Failures	0%
Avg response	29,614ms
Max response	56,073ms
Throughput	0.33 req/s

Phase 2 Analysis:

Payment processor speed: 1 order per 3 seconds = 0.33 orders/sec

Flash sale demand: ~60 orders/second

Orders lost per second: ~59.67 orders/second

Only 19 requests were processed in 60 seconds despite 20 concurrent users. The system did not drop requests but simply made customers wait up to 56 seconds, which is functionally equivalent to failure from a user experience perspective.

Phase 3: Async Solution

Flash Sale Async Test (20 users, 60s)

```
(base) emilychen@emilys-Air-2 locust % TEST_MODE=async locust -f locustfile.py \
--host=http://CS6650L2-alb-1613587836.us-west-2.elb.amazonaws.com \
--users=20 --spawn-rate=10 --run-time=60s --headless
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
Aggregated		3194	0(0.00%)	33	22	168	29	59.40	0.00
POST	POST /orders/async	3307	0(0.00%)	33	22	168	29	59.70	0.00
Aggregated		3307	0(0.00%)	33	22	168	29	59.70	0.00
POST	POST /orders/async	3422	0(0.00%)	33	22	168	29	59.60	0.00
Aggregated		3422	0(0.00%)	33	22	168	29	59.60	0.00

[2026-03-13 23:02:43,520] emilys-Air-2/INF0/locust.main: --run-time limit reached, shutting down
[2026-03-13 23:02:43,574] emilys-Air-2/INF0/locust.main: Shutting down (exit code 0)

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	POST /orders/async	3524	0(0.00%)	33	22	168	29	58.89	0.00
Aggregated		3524	0(0.00%)	33	22	168	29	58.89	0.00

Response time percentiles (approximated)

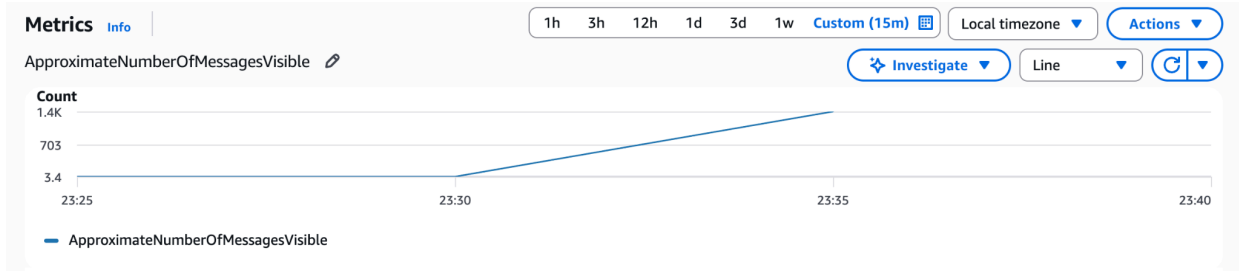
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%
POST	POST /orders/async	29	31	35	38	48	56	69	88	160	17
Aggregated		29	31	35	38	48	56	69	88	160	17

Metric	Sync	Async
Requests	19	3,524
Failures	0%	0%
Avg response	29,614ms	33ms
Max response	56,073ms	168ms
Throughput	0.33 req/s	58.89 req/s

The async endpoint accepted **185x more orders** (3,524 vs 19) with average response time dropping from 29,614ms to 33ms. By publishing to SNS and returning 202 immediately, the API decoupled order acceptance from payment processing.

Phase 4: Queue Buildup Problem

With the async endpoint accepting ~59 orders/sec but a single worker processing only 0.33/sec, messages accumulated rapidly in SQS.



Order acceptance rate: 58.89 orders/sec
Single worker rate: 0.33 orders/sec
Queue growth rate: ~58.56 messages/sec
Peak queue depth: 3,531 messages
Time to clear backlog: $3,531 \div 0.33 = \sim 178$ minutes

Customers received immediate 202 responses but waited nearly 3 hours for order confirmation — hence the customer service calls..

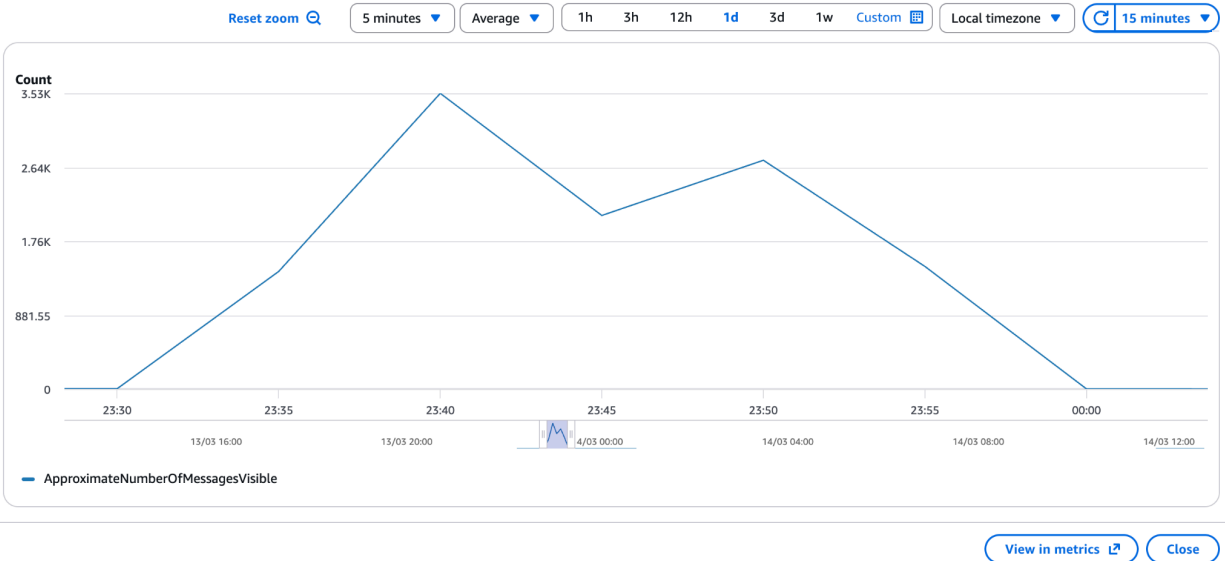
Phase 5: Worker Scaling

We scaled goroutines within a single ECS task (256 CPU, 512MB) to find the minimum workers needed to prevent queue buildup at 58.89 orders/sec.

Theoretical minimum:

workers = incoming rate $\times$ processing time
= 58.89 orders/sec $\times$ 3 seconds
= ~177 workers minimum

Workers	Processing Rate	Peak Queue Depth	Time to Drain
1	0.33/sec	3,531	~178 min
5	1.67/sec	2,217	~22 min
20	6.67/sec	1,768	~4.4 min
100	33.3/sec	1,253	~38 sec
200	66.7/sec	~0	instantly



The CloudWatch graph shows the complete Phase 5 experiment timeline. Between 23:30–23:40, the queue spiked to 3.53K messages with insufficient workers — clear evidence that 1–20 workers cannot keep pace with ~59 orders/sec. Between 23:45–23:50, a secondary spike to 2.64K reflects continued testing with 20 and 100 workers, both still falling behind. After 23:50, with 200 goroutines processing at 66.7 orders/sec, the queue drained steadily to zero by midnight and remained flat — confirming that 200 workers successfully absorbed the flash sale load with a ~13% safety buffer over the theoretical minimum of 177.

Goroutines are extremely lightweight in Go (~8KB stack each), so 200 goroutines consumed only ~1.6MB memory — well within our 512MB task limit. This demonstrates that vertical scaling within a single container is a viable and cost-effective strategy before resorting to horizontal scaling across multiple ECS tasks.

Analysis Questions

Q1: How many times more orders did async accept vs sync?

$3,524 \div 19 = \sim 185x$ more orders accepted with the async approach.

Q2: What causes queue buildup and how do you prevent it?

Queue buildup occurs when the message processing rate is lower than the message arrival rate. In our case, a single worker processing at 0.33/sec could not keep up with 58.89 orders/sec incoming, causing ~58.56 messages/sec to accumulate. Prevention strategies include scaling workers to match load (minimum 177 for our workload), auto-scaling based on

queue depth metrics in CloudWatch, reducing processing time per message, or switching to serverless processing such as Lambda which scales automatically per message.

Q3: When would you choose sync vs async in production?

Choose **sync** when the result is needed immediately (bank balance check, inventory availability), operations complete quickly (<100ms), or strong consistency is required.

Choose **async** when processing is slow (>1s), load is spiky or unpredictable, the user can tolerate eventual completion (order confirmation, email notifications), or decoupling producers from consumers improves resilience.

Part III: Serverless with Lambda

Architecture

Replacing ECS workers with Lambda eliminates SQS entirely:

Before: API → SNS → SQS → ECS Workers (you manage everything)

After: API → SNS → Lambda (AWS manages everything)

Cold Start Observations

▼	2026-03-14T12:51:34.455-07:00	2026/03/14 19:51:34 [lambda] processing order f8713c8e-378e-4543-882b-acab2f6fbc6a for customer 1	
	2026/03/14 19:51:34	[lambda] processing order f8713c8e-378e-4543-882b-acab2f6fbc6a for customer 1	📄
▼	2026-03-14T12:51:37.458-07:00	2026/03/14 19:51:37 [lambda] completed order f8713c8e-378e-4543-882b-acab2f6fbc6a	
	2026/03/14 19:51:37	[lambda] completed order f8713c8e-378e-4543-882b-acab2f6fbc6a	📄
▼	2026-03-14T12:51:37.458-07:00	[lambda] order f8713c8e-378e-4543-882b-acab2f6fbc6a status: completed	
		[lambda] order f8713c8e-378e-4543-882b-acab2f6fbc6a status: completed	📄
▼	2026-03-14T12:51:37.459-07:00	END RequestId: 639e4735-793f-4c74-a421-f53e6eadd9de	
		END RequestId: 639e4735-793f-4c74-a421-f53e6eadd9de	📄
▼	2026-03-14T12:51:37.459-07:00	REPORT RequestId: 639e4735-793f-4c74-a421-f53e6eadd9de Duration: 3006.76 ms Billed Duration: 4367 ms Memory Size:...	
		REPORT RequestId: 639e4735-793f-4c74-a421-f53e6eadd9de Duration: 3006.76 ms Billed Duration: 4367 ms Memory Size: 512 MB Max Memory Used: 23 MB Init Duration: 1359.72 ms	📄
No newer events at this moment. Auto retry paused. Resume			Back to top ^

REPORT RequestId: ac367463-6e29-4dde-9e7b-7145b8f57213	Duration: 3004.18 ms	Billed Duration: 3083 ms	Memory Size: 512 MB	
Max Memory Used: 23 MB	Init Duration: 78.07 ms			
2026-03-14T12:51:39.286-07:00	START RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba	Version: \$LATEST		
	START RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba	Version: \$LATEST		
2026-03-14T12:51:39.286-07:00	2026/03/14 19:51:39 [lambda]	received SNS message: {"order_id":"58aa58f1-98f8-4f78-bdf1-b5f2464ec04a","customer_i...		
2026-03-14T12:51:39.286-07:00	2026/03/14 19:51:39 [lambda]	processing order 58aa58f1-98f8-4f78-bdf1-b5f2464ec04a for customer 7		
2026-03-14T12:51:42.289-07:00	[lambda]	order 58aa58f1-98f8-4f78-bdf1-b5f2464ec04a status: completed		
2026-03-14T12:51:42.289-07:00	2026/03/14 19:51:42 [lambda]	completed order 58aa58f1-98f8-4f78-bdf1-b5f2464ec04a		
2026-03-14T12:51:42.290-07:00	END RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba			
2026-03-14T12:51:42.290-07:00	REPORT RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba	Duration: 3003.59 ms Billed Duration: 3004 ms Memory Size:...		
	REPORT RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba	Duration: 3003.59 ms Billed Duration: 3004 ms Memory Size: 512 MB		
Max Memory Used: 23 MB				
2026-03-14T12:51:42.535-07:00	START RequestId: a8c08e4f-0918-45eb-9aeb-88c43acea080	Version: \$LATEST		
026-03-14T12:51:42.290-07:00	END RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba			
026-03-14T12:51:42.290-07:00	REPORT RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba	Duration: 3003.59 ms Billed Duration: 3004 ms Memory Size:...		
	REPORT RequestId: dcca0329-2a7d-47f4-afee-bd7be99d5dba	Duration: 3003.59 ms Billed Duration: 3004 ms Memory Size: 512 MB		
Max Memory Used: 23 MB				
026-03-14T12:51:42.535-07:00	START RequestId: a8c08e4f-0918-45eb-9aeb-88c43acea080	Version: \$LATEST		
026-03-14T12:51:42.535-07:00	2026/03/14 19:51:42 [lambda]	received SNS message: {"order_id":"9293b94d-4763-4fd1-a913-a8df1dd5dc08","customer_i...		
026-03-14T12:51:42.535-07:00	2026/03/14 19:51:42 [lambda]	processing order 9293b94d-4763-4fd1-a913-a8df1dd5dc08 for customer 10		
026-03-14T12:51:45.537-07:00	2026/03/14 19:51:45 [lambda]	completed order 9293b94d-4763-4fd1-a913-a8df1dd5dc08		
026-03-14T12:51:45.537-07:00	[lambda]	order 9293b94d-4763-4fd1-a913-a8df1dd5dc08 status: completed		
026-03-14T12:51:45.539-07:00	END RequestId: a8c08e4f-0918-45eb-9aeb-88c43acea080			
026-03-14T12:51:45.539-07:00	REPORT RequestId: a8c08e4f-0918-45eb-9aeb-88c43acea080	Duration: 3003.22 ms Billed Duration: 3004 ms Memory Size:...		

Invocation	Duration	Init Duration	Type
1st	3,006ms	1,359ms	Cold (container pull)
2nd	3,004ms	545ms	Cold
3rd	3,004ms	78ms	Cold (image cached)
4th	3,003ms	85ms	Cold
5th+	~3,003ms	none	Warm

Cold starts occurred on the first 4 invocations as Lambda spun up parallel containers. After the image was cached, cold start overhead dropped to ~78ms. Warm invocations showed exactly 3,000ms — purely our payment processing delay.

Cold start impact: $78\text{ms} \div 3,000\text{ms} = 2.6\%$ overhead — negligible for 3-second payment processing.

Cost Analysis

Current ECS Cost:

2 ECS tasks $\times$ \$8.50/month = \$17/month (always running)

Lambda Cost (10,000 orders/month):

Requests: 10,000 (under 1M free tier) = \$0

GB-seconds: $10,000 \times 3s \times 0.5GB = 15,000$ (under 400K free tier) = \$0

Total: \$0/month

Break-even point:

Free tier covers ~267K orders/month

Lambda stays cheaper than ECS until ~1.7M orders/month

Switch Recommendation

Based on our observations, we should switch to Lambda for order processing. Cold starts occurred only on the first invocation or after extended idle periods, adding ~78ms overhead on 3-second processing — a negligible 2.6% impact that customers would never notice. The cost advantage is compelling: Lambda costs \$0 for under 267K orders/month compared to \$17/month for always-running ECS tasks. Most importantly, Lambda eliminates all operational overhead — no SQS queue depth monitoring, no manual worker scaling, and no 3am alerts when the queue spikes. The trade-off of losing SQS retry guarantees is acceptable at our current scale since SNS retries twice before discarding, which is sufficient given the API already returns 202 immediately. Lambda is the clear architectural choice until we exceed 267K orders/month or require advanced retry, replay, or batch processing capabilities that SQS provides.